Auditable Bounded Deliberation on Constrained Game Hardware: A Checkers Case Study

Christopher A. Tucker
cartheur@pm.me

Draft — November 2026

Abstract

This paper studies whether a small deterministic controller can look ahead, choose an action, and report that action under fixed memory and timing limits without hiding its reasoning in one large game loop. Checkers is the case study, not the claimed contribution. The controller has named stages for receiving a board, checking it, generating moves, bounded anticipation, evaluation, decision, and response. A portable C++11 core, mailbox protocol, board-geometry tests, and initial rules fixtures provide the starting artifact. This draft makes no claim of strong play, complete Checkers rules, or novelty. Its proposed contribution is conditional on later evidence: complete traces, measured bounds, cross-platform replay, and comparison with a conventional implementation.

1 Introduction

Classic game systems show that a game player can be small and deterministic. This project asks a narrower systems question: can a controller that looks ahead and chooses a move remain inspectable and testable while running on a desktop emulator and small target hardware?

The proposed answer is an FSA controller with bounded working memory, named state, limited work per step, and visible mailbox results. Each stage can be tested on its own instead of hiding the whole process inside one search routine.

2 Research question and publication gate

The research question is:

> Can a finite-state controller with bounded working memory perform useful look-ahead and deterministic action selection under constrained timing, while leaving a replayable record from input board to published action?

Checkers supplies a concrete, finite, perfect-information case in which the question can be tested. It is not sufficient to show that the program plays a legal game. Before this work is presented as a research paper, it must provide all of the following:

1. **An operational contract.** State what `ANTICIPATE` may read and change, what `DECISION` guarantees, and when a mailbox response becomes visible.

2. **Checked properties.** Show that every published move is legal, deterministic, associated with a bounded projected line, and written as a complete response rather than partial data.

3. **Measured bounds.** Report time per FSA step, time per complete decision, code size, working memory, and the limit on retained projection data.

4. **Cross-platform replay.** Run the same fixtures and traces through the desktop core, Mira, and the intended hardware path.

5. **A fair comparison.** Compare the design with a simple, conventional game loop on defect localization, reproducibility, memory, and timing. Playing strength is a secondary measure.

If these conditions are not met, this document should remain a technical report. A paper that merely presents another Checkers program is not the goal.

3 Architecture

The controller starts in q_0 = WAIT_REQUEST. It reads and checks a board, checks the result of the previous turn, and generates legal moves. It then enters q_4 = ANTICIPATE to project candidate lines before scoring them. A chosen move enters q_7 = DECISION; the next transition writes the response. Bad input or a terminal position enters q_E = ERROR; the next transition writes an error response. The source repository contains the state diagram and the formal input and output alphabets.

The FSA design is explicitly based on the treatment of finite-automaton behavior and synthesis by Trakhtenbrot and Barzdin [5]. The book supplies the starting discipline: state, stored information, inputs, outputs, and transitions must all be stated. This project is not presented as a new theorem in automata theory.

> **Current evidence artifact:** `research/checkers-evidence/FSA-LOGIC.md`. The rendered diagram is `FSA-LOGIC.png`. At this draft stage the portable core implements the request/response skeleton; move generation provides evidence for the next generation state.

3.1 Separation of control and policy

The FSA is not the game strategy. It says which step comes next, what data is kept, and how errors are reported. `ANTICIPATE` is a state; projection is the work done in that state. `DECISION` and `ERROR` are also states; their outgoing transitions write to the mailbox. The strategy is a separate deterministic rule for ranking legal projected moves. It can change without changing the protocol or the rules checks.

4 Anticipation and decision

The key distinction is between looking at the consequences of a move and choosing the move to send to the game surface. Keeping those jobs separate makes a small player easier to test and explain.

4.1 `ANTICIPATE`: bounded projection, not commitment

The `ANTICIPATE` state takes one legal candidate and builds a private board position for it. At first, this means applying the move and recording captures, promotion, whose turn it is, and any required jump continuation. Later it may also look at a limited opponent reply and one reply after that. It does not predict what an opponent will do. It simply builds the positions that follow from a stated candidate line.

Three rules keep this work clear and small:

1. **Start with a legal move.** This state uses only moves produced by the rules engine.
2. **Keep the work private and limited.** The accepted board does not change. A scratch board and a trace are used instead. One FSA step does only a small unit of work, such as applying one move or one continuation.
3. **Keep the trace.** The trace records the candidate, projected board, captures, continuations, and later score. It shows why a move could be chosen and helps check the next board that arrives.

The state loops until it has made every projection requested for this turn. It does not stop merely because it has found a move that looks good. This gives the ranking rule a known set of inputs and keeps look-ahead from taking over the bus or the frame time.

4.2 `DECISION`: one move is made final

`DECISION` begins after every candidate has a score and `SELECT` has chosen one using a fixed ordering. It does not score moves again. It holds the chosen move and its trace until the result is written to the mailbox.

The transition out of `DECISION` writes the move data and sets `READY` last. A reader therefore cannot see a ready result before the move is complete. Each reported move can be traced back to its request, a legal candidate, its projected line, and the comparison that chose it.

Tie-breaking also belongs here. If two candidates have the same value, a fixed ordering—for example origin square followed by landing order—chooses one without randomness. The same input then gives the same result on the host and on the target, which helps testing and replay.

4.3 Why the separation matters

Without this split, a game program can mix up three questions: is a move legal, what follows from it, and should it be chosen? The FSA gives these to `GENERATE`, `ANTICIPATE`, and `SELECT`/`DECISION`. This does not guarantee strong play. It does make the source of a choice testable. A better ranking rule can be added without changing the rules, the limits on projection, or the mailbox order.

4.4 Portable boundary

The shared core is C++11 and has no target-specific dependencies. The host emulator and target firmware sit around the same board, mailbox, and agent interfaces. Hardware measurements are not yet available.

5 Validation methodology

Validation proceeds in layers:

1. Verify the 32-square board mapping, coordinate round trips, edge behavior, and invalid encodings with independent geometry fixtures.
2. Validate mailbox request parsing, sequencing, and error responses.
3. Fixture-test legal move generation and move application.
4. Add deterministic policy tests over fixed board positions.
5. Evaluate complete players against declared baselines using a documented, reproducible protocol.

Only the first three layers are partially implemented in this draft. The results section will remain intentionally incomplete until later layers and their measurements exist.

6 Current scope and limitations

The current rules implementation covers uncrowned-men quiet moves and single captures with compulsory-capture filtering. Promotion, king movement, multi-jumps, draw rules, policy, deeper projection, target adapters, and performance experiments remain future work. These limits are deliberate boundaries, not claims of completed functionality.

7 Literature survey and position

7.1 Automata and controller synthesis

The direct foundation is the treatment of automaton behavior and synthesis by Trakhtenbrot and Barzdin [5]. It supplies the discipline used here: state, stored information, inputs, outputs, and transitions are stated rather than left implicit.

Recent reactive-synthesis work addresses a different but related problem: constructing a controller from a formal specification. Geatti, Montali, and Rivkin construct finite, terminating controllers for finite-trace specifications through symbolic automata [1]. Their result is not a model of this Checkers implementation, which is deliberately specified and fixture-tested by hand. It is useful as a comparison point: formal synthesis can provide specification-level guarantees, while this project must show that its explicit controller, rules, and mailbox interface work together on its target systems.

The SYNTCOMP reports provide a second methodological comparison point. Jacobs et al. compare synthesis tools through shared benchmarks, reproducible runs, and measurements of solution size and quality [2]. The present project should adopt that spirit on a smaller scale: publish the positions, traces, build revision, state count, memory use, decision latency, and error cases used for each result.

7.2 Finite machines as game strategies

Černý, Bošanský, and An study finite-state machines as compact strategy representations for extensive-form games [6]. They show that restricting strategy representation to machines can

reduce the search space and support scalable algorithms. Their setting is theoretically richer than the one here: it concerns extensive-form games, imperfect information, and equilibrium computation. The present work is instead a perfect-information Checkers player with a deliberately small controller and a separate move ranking rule.

This difference makes the comparison productive rather than competitive. A future result could be stronger in an engineering sense if it demonstrates an end-to-end, reproducible machine strategy on constrained hardware: legal rules, bounded anticipation, deterministic decision, atomic mailbox response, and replayable traces. It would not be stronger than their equilibrium results unless it also establishes the corresponding game-theoretic guarantees.

7.3 Checkers and game-engine testing

Checkers itself was weakly solved by Schaeffer et al. in 2007: with perfect play, the game is a draw [4]. This is a necessary historical reference and a limit on the claims of this paper. The goal is not to solve Checkers again or to compete with a large solver. It is to build a transparent opponent whose behavior can be followed from an input board to one published move.

Testing is therefore as important as playing strength. Méndez et al. use metamorphic testing to find inconsistencies in chess-engine evaluation [3]. Their work suggests a useful next step for this project: add Checkers-specific metamorphic tests for board reflection, rotation where rule conventions permit it, side exchange, and equivalent position encodings. Such tests complement ordinary legal-move fixtures and can reveal errors even when no trusted best move is available.

7.4 Claim and comparison plan

Finite-state controllers, finite-state game strategies, reactive synthesis, and strong Checkers play are all established areas. The possible contribution of this project is narrower: a documented bridge from a finite-state controller to a complete, portable Checkers surface, with rules, bounded anticipation, decision publication, and tests visible at every boundary.

The claim remains provisional until it is measured. The paper will compare the completed player against declared deterministic baselines and report rule coverage, metamorphic-test results, state-by-state timing, code and memory size, and replayable traces on both desktop and target systems. Those results will show whether the design has a useful advantage in clarity, reproducibility, or resource use. They must not be replaced by a general claim of stronger game play or stronger formal guarantees.

8 Candidate contribution and possible impact

The work does not yet claim novelty. Finite automata can play games, and Checkers can be solved; both are established. The candidate contribution is an end-to-end implementation method for a small game player: one that carries an explicit finite-state controller from desktop tests to a constrained target while keeping its rules, bounded look-ahead, decision publication, and hardware costs visible.

Four parts of the design matter together:

1. **Anticipation as a named state.** `ANTICIPATE` makes bounded projection visible. It is neither a hidden search loop nor a commitment to a move.

2. **Decision as a named state.** `DECISION` separates choosing a move from writing it to the mailbox. The following transition writes the complete move before it marks the result ready.

3. **One portable core.** The same deterministic rules and controller interfaces are intended to run in desktop tests, the Mira emulator, the CP1610 game surface, and the Teensy-assisted cartridge system.
4. **An evidence trail.** State diagrams, mailbox fixtures, rule fixtures, projected traces, resource measurements, and hardware observations can be checked separately and kept with the source.

If the completed system shows that these parts remain consistent across the host and target, it may offer a useful method for other small game surfaces and constrained interactive systems. The claim must be earned with the evidence listed in the publication gate. Better Checkers play is a separate empirical question, not the central claim.

9 Evaluation plan

The completed paper will report:

- rules conformance against a documented position suite;
- determinism and protocol-error behavior;
- move quality against explicitly named baseline policies;
- memory, code size, and latency on each supported platform; and
- limitations, failure cases, and reproducibility instructions.

10 Conclusion

The project uses Checkers to test a controller that separates rules, bounded anticipation, decision, and publication. This draft records the design and the evidence available so far. It becomes a research paper only if the required cross-platform, bounded, and comparative evidence is produced.

References

[1] Luca Geatti, Marco Montali, and Andrey Rivkin. Reactive synthesis for DECLARE via symbolic automata. *arXiv preprint arXiv:2212.10875*, 2022.

[2] Swen Jacobs, Guillermo A. Pérez, Remco Abraham, Véronique Bruyère, et al. The reactive synthesis competition (SYNTCOMP): 2018–2021. *International Journal on Software Tools for Technology Transfer*, 26:551–567, 2024.

[3] Manuel Méndez, Miguel Benito-Parejo, Alfredo Ibias, and Manuel Núñez. Metamorphic testing of chess engines. *Information and Software Technology*, 162:107263, 2023.

[4] Jonathan Schaeffer, Neil Burch, Yngvi Björnsson, Akihiro Kishimoto, Martin Müller, Robert Lake, Paul Lu, and Steve Sutphen. Checkers is solved. *Science*, 317(5844):1518–1522, 2007.

[5] B. A. Trakhtenbrot and Ya. M. Barzdin. *Finite Automata: Behavior and Synthesis*. North-Holland Publishing Company, 1973. Translated from the Russian by D. Louvish; English translation edited by E. Shamir and L. H. Landweber.

[6] Jakub Černý, Branislav Bošanský, and Bo An. Finite state machines play extensive-form games. In *Proceedings of the 21st ACM Conference on Economics and Computation*, 2020.